Pretraining, Autoencoders, CNNs, and RNNs

Duration: 1 Hour

Target Audience: Data Science/Computer Science students

Prerequisites: Understanding of basic deep learning concepts, backpropagation, neural network fundamentals

Advanced Deep Learning Architectures

Making Deep Learning Practical

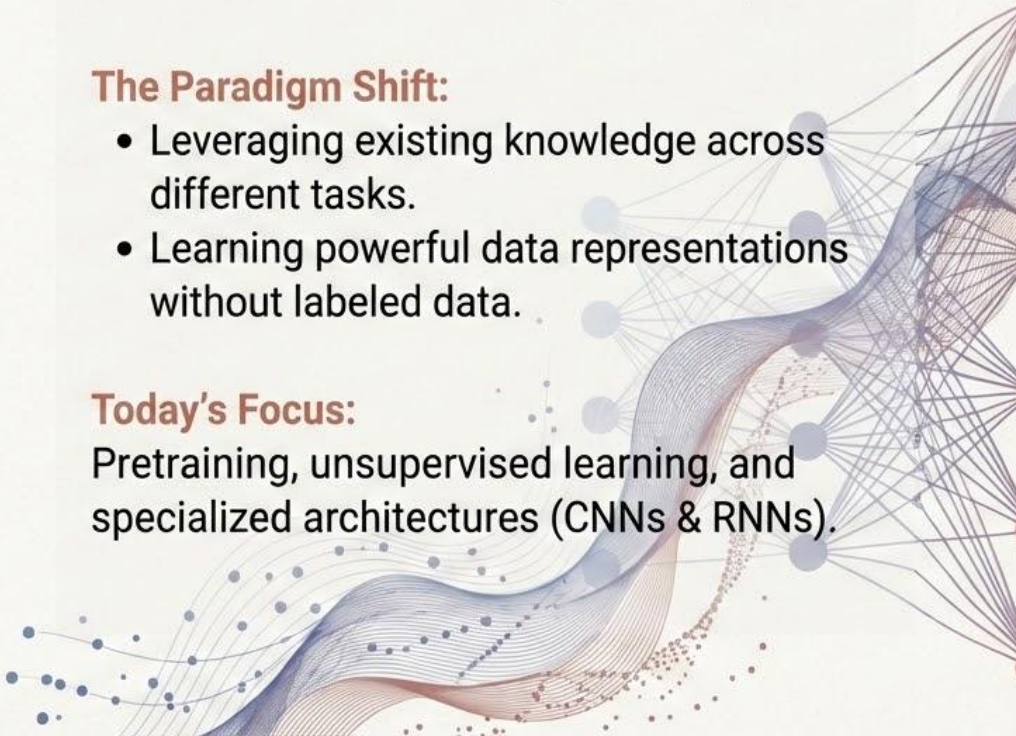
The Reality of Deep Learning: Training networks from scratch demands massive amounts of data and computational power.

The Paradigm Shift:

- Leveraging existing knowledge across different tasks.
- Learning powerful data representations without labeled data.

Today's Focus:

Pretraining, unsupervised learning, and specialized architectures (CNNs & RNNs).



Learning Objectives

What You Will Master

By the end of this session, you will be able to:

- Understand pretraining and transfer learning in deep networks.
- Master the mechanics of cross-entropy loss for classification tasks.
- Learn how autoencoders are used for unsupervised representation learning.
- Understand Convolutional Neural Networks (CNNs) for processing image data.
- Explore Recurrent Neural Networks (RNNs) for handling sequence data.

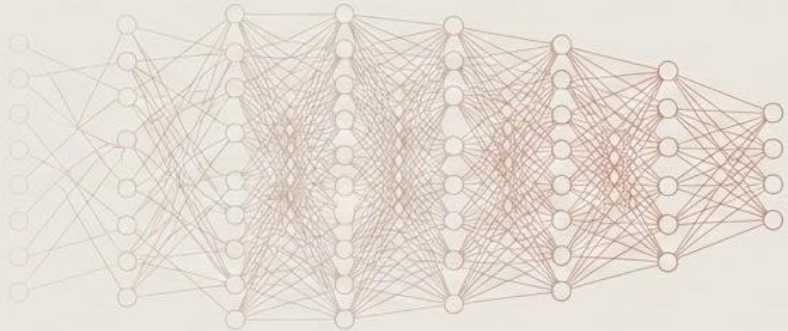
The Challenge of Deep Networks

Why Starting from Scratch is Difficult

The Problem:

Training very deep neural networks entirely from scratch (with randomized weights) is notoriously difficult.

- It requires massive, carefully labeled datasets.
- The training process is highly unstable and computationally expensive.



The Solution: Pretraining

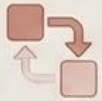
* Initialize the network's weights using foundational knowledge already learned from a different, related task before training it on your specific target task.

Approaches to Pretraining

From Greedy Layerwise to Transfer Learning

1. Greedy Layerwise Pretraining (Historical Approach: 2006-2010)

- **How it worked:** Train the network one single layer at a time, starting from the input. Each layer learns to represent the output of the previous layer.
- **Status:** Largely obsolete today due to better weight initialization techniques and batch normalization.



2. Transfer Learning & Fine-Tuning (The Modern Standard)

How it works:

1. Take a model already trained on a massive dataset (e.g., ImageNet).
2. Strip away its final classification layer.
3. Attach a new layer designed for your specific task.
4. "Fine-tune" the weights using your smaller, target dataset.

- **Status:** The modern standard and widely adopted technique.

The Architecture of Transfer Learning

Recycling AI Knowledge

14 million general web images

Source Task

Pre-trained Model

Extract the foundational weights (The "Knowledge")

Architecture Adjustment

Replace the final output layer to match the new, specific categories.

5,000 medical X-rays

Target Task (Fine-Tuning)

Train the adjusted model on a small, domain-specific dataset

Why We Pretrain

The Benefits and Real-World Impact

Less Data, Better Results

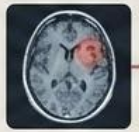
The Core Benefits

Data Efficiency: Requires drastically less labeled data to achieve high accuracy.

Speed: Faster mathematical convergence during training.

Performance Margin: Significantly better final performance, particularly on small datasets where training from scratch would cause severe overfitting.

Real-World Examples



Computer Vision: Using ImageNet-pretrained ResNet models to detect tumors in medical imaging.

NLP: Using BERT (pretrained on the entirety of Wikipedia) to analyze customer sentiment.

Speech: Pretraining on a massive general speech corpus before fine-tuning for specific regional accents.

Evaluating Classification Models

Why Mean Squared Error (MSE) Fails and Cross-Entropy Succeeds

The Historical Default (Regression):

Mean Squared Error (MSE) is the gold standard for predicting continuous numbers.

Regression (Linear Errors)

The Problem: MSE Fails for Classification

- MSE assumes errors are linear.
- MSE does not penalize "confident wrongness" heavily enough.

Confident Wrongness: 99% certain it's a cat, but it's a dog. MSE doesn't capture the severity.

The Solution: Cross-Entropy Loss

Specifically designed to measure the difference between a predicted probability distribution and the true probability distribution.

Binary Cross-Entropy

Classification for Two Outcomes

Used when the model must choose between exactly two classes (e.g., Spam vs. Not Spam).

The Formula:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Variable Definitions:

- y: The true label ($1 for True, $0 for False).
- $\hat{y}$: The model's predicted probability (between $0.0 and $1.0).

How it works:

Because y is either 0 or 1, one half of this equation always cancels out, leaving only the logarithmic penalty for the actual truth.

Scenario 1: True Label $y = 1$

1 → Penalty: $-\log(\hat{y})$

Penalty: $(1-1)\log(1-\hat{y})$

Second term becomes 0

Scenario 2: True Label $y = 0$

0 → Penalty: $0 \cdot \log(\hat{y})$

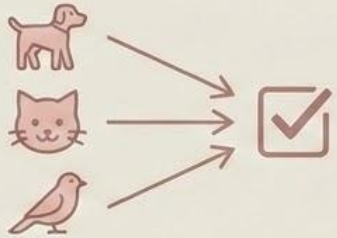
Penalty: $-\log(1-\hat{y})$

First term becomes 0

Categorical Cross-Entropy

Classification for Many Outcomes

Used when the model must choose between three or more classes (e.g., Dog, Cat, Bird).



The Formula:

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Variable Definitions:

- C: The total number of classes.
- y_i: The true label (uses one-hot encoding: 1 for the true class, 0 for all others).
- $\hat{y}_i$: The predicted probability for class i.

$$[0, 1, 0]$$

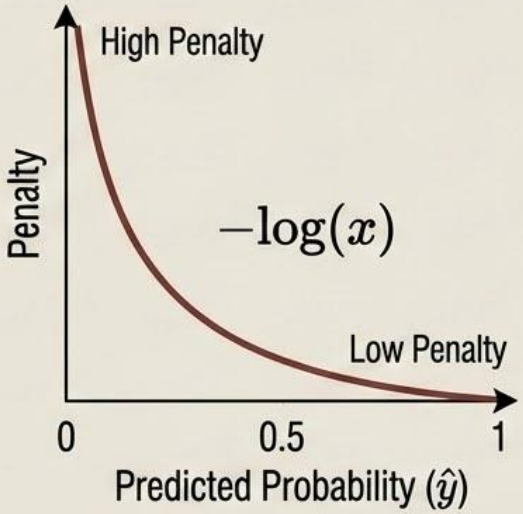
Cat (True Class)

Why Cross-Entropy Works

Penalizing Confidence

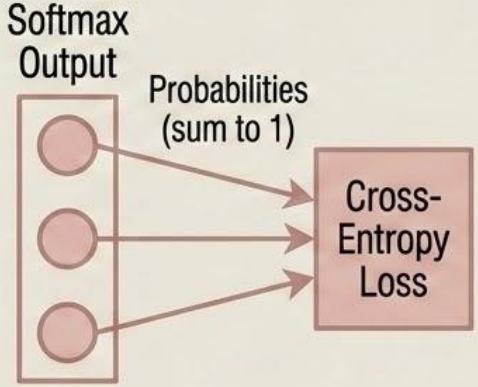
The Logarithmic Penalty

The negative log function creates an exponential penalty for confident wrong predictions.



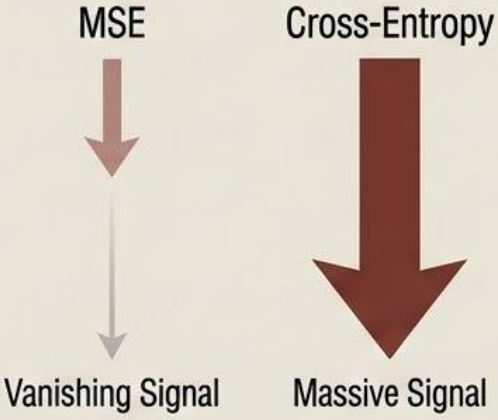
Natural Synergy

It works perfectly with the 'Softmax' activation function used in classification output layers.



Non-Vanishing Gradients

Unlike MSE, the gradients for cross-entropy do not vanish when the prediction is completely wrong, ensuring the model receives a massive signal to correct its mistake.

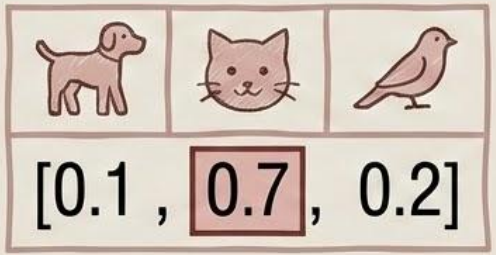


Cross-Entropy in Action

A Practical Example

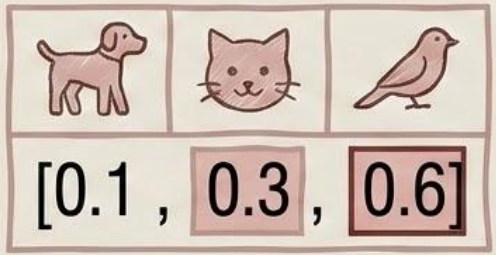
True class (One-Hot): [0, 1, 0] (Cat)

Scenario A: The model is somewhat unsure, but mostly right.



Loss = –log(0.7) = **0.357**

Scenario B: The model is confidently wrong (thinks it's a bird).



Loss = –log(0.3) = **1.204**

(Over 3x higher penalty!)

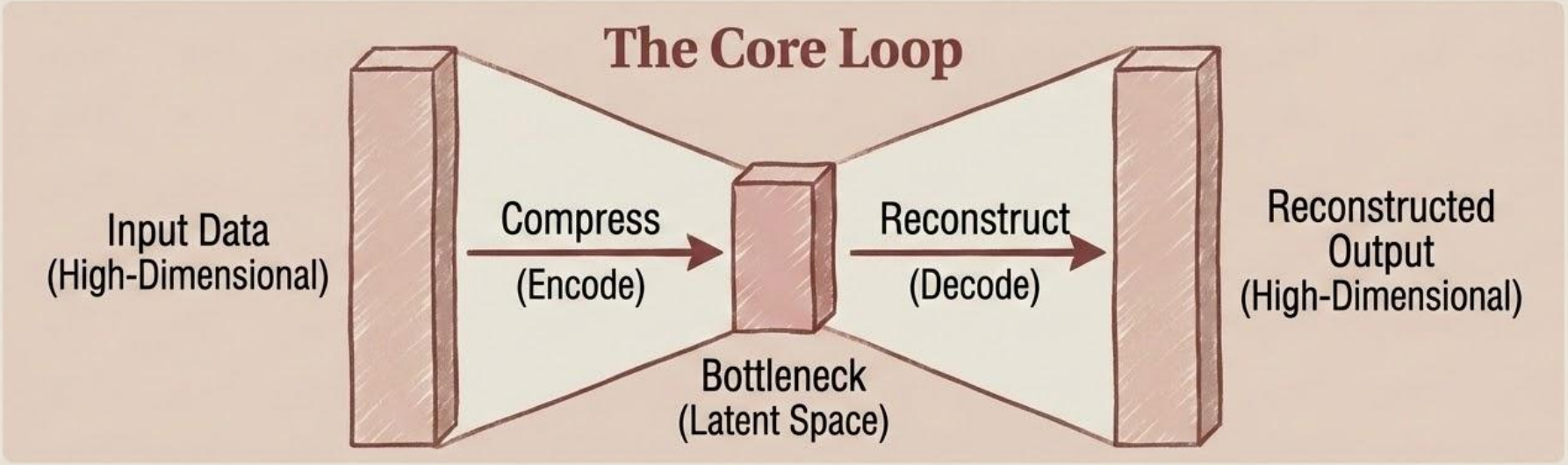
Unsupervised Deep Learning

The Autoencoder

Learning by Compression and Reconstruction

The Paradigm Shift: Most networks we've discussed are supervised (they require labeled data). Autoencoders are unsupervised—they learn from the data itself.

Definition: A neural network designed to learn efficient, low-dimensional representations of data by forcing it through a bottleneck.



Anatomy of an Autoencoder

The Architecture

The Encoder

A neural network that compresses the input into a mathematical representation.

The Latent Space (Bottleneck)

The physical constraint in the middle of the network. It forces the network to throw away useless data and keep only the most salient features.

The Decoder

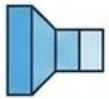
A neural network that takes the compressed representation (z) and attempts to rebuild the original input.

The Training Objective

Minimize the Reconstruction Error (e.g., the Mean Squared Error between the original input x and the reconstruction $\hat{x}$).

Types of Autoencoders

Beyond Simple Compression



Undercomplete

The bottleneck dimension is strictly smaller than the input.

Primary Use Case

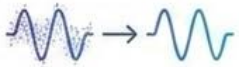
Standard dimensionality reduction

Overcomplete

The bottleneck is larger than the input (requires heavy mathematical regularization to prevent simple copying).

Primary Use Case

Complex feature extraction

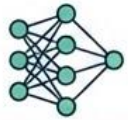


Denoising

The network is fed corrupted/noisy data but scored on reconstructing the clean original.

Primary Use Case

Removing static/noise from audio or images

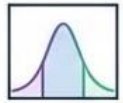


Sparse

Adds a penalty that forces most neurons in the bottleneck to stay inactive (output zero).

Primary Use Case

Advanced feature discovery



Variational (VAE)

Instead of fixed numbers, the latent space learns probability distributions (means and variances).

Primary Use Case

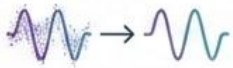
Generative AI (creating entirely new data)

Real-World Impact

How the Industry Uses Unsupervised Learning

Dimensionality Reduction

Acting as a highly powerful, non-linear version of PCA (Principal Component Analysis) to compress data for faster storage and processing.

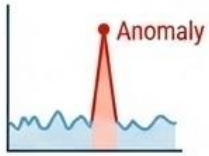


Denoising

Automatically cleaning up old audio files, low-light photographs, or corrupted medical scans.

Feature Learning

Pre-training the weights of an encoder on massive unlabeled datasets before transferring them to a supervised task.



Anomaly Detection

- Mechanism: Train an autoencoder exclusively on normal data. When fed an anomaly (like credit card fraud), it won't know how to reconstruct it.
- Result: A massive spike in reconstruction error instantly flags the anomaly.

Advanced **Training** Techniques

Keeping Deep Networks on Track

The Reality: Even with ReLU, Adam, and Dropout, training networks with dozens or hundreds of layers is mathematically chaotic.

The Final Toolkit: We rely on four essential techniques to stabilize the math, control the speed, and guarantee convergence:

- **Batch Normalization**
- **Residual Connections (ResNets)**
- **Learning Rate Warmup**
- **Gradient Clipping**

Batch Normalization

Solving Internal Covariate Shift

The Problem & Solution

The Problem: Data distribution into deeper layers constantly changes as early layers update (**Internal Covariate Shift**). Deeper layers chase a moving target.

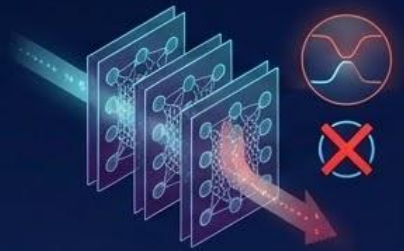
The Solution: Batch Normalization. Mathematically forces layer inputs to zero mean, unit variance.

The Benefits

- Drastically speeds up training (allows for much higher learning rates).
- Smooths the optimization landscape.
- Acts as a mild regularizer, slightly reducing the need for heavy Dropout.

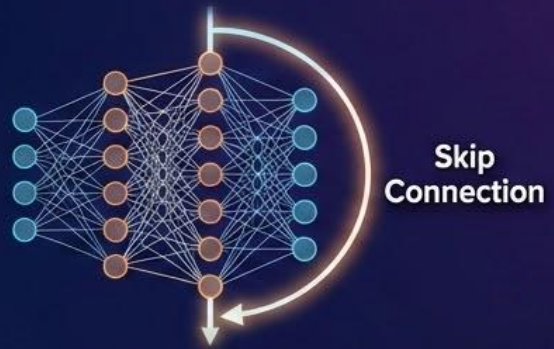
Residual Connections (ResNets)

The Deep Learning Expressway



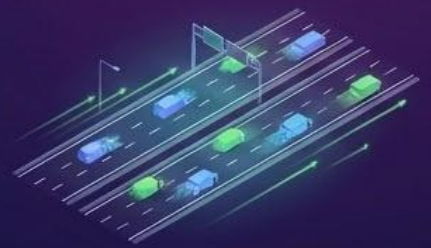
The Problem

In extremely deep networks, even ReLU can't fully stop the learning signal (gradient) from degrading as it passes back through 50 or 100 sequential layers.



The Solution: Skip Connections

Create physical "shortcuts" that allow data and gradients to bypass one or more layers entirely. Formally, instead of learning a direct mapping $H(x)$, the layer learns a residual $F(x)$, and the output becomes $F(x) + x$.



The Result

Enables the successful training of incredibly deep networks (100+ to 1,000+ layers) by giving gradients an unobstructed "expressway" to flow backward.

Warmup and Clipping

Managing the Extremes

Learning Rate Warmup

Mechanism: Start training with an extremely small learning rate and gradually increase it over the first few thousand steps.

Why: At step zero, network weights are entirely random. Taking a massive learning step immediately will heavily destabilize the model.

Standard in: Transformer architectures (like ChatGPT).

Gradient Clipping

Mechanism: Establish a strict maximum value threshold. If a calculated gradient exceeds this threshold, forcefully scale it down (clip it).

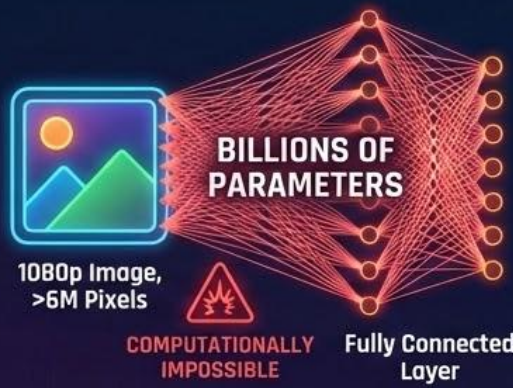
Why: Completely eliminates the “Exploding Gradient” problem by ensuring no single update can violently destroy the network’s weights.

Standard in: Recurrent Neural Networks (RNNs) and Transformers.

The Problem with Fully Connected Layers

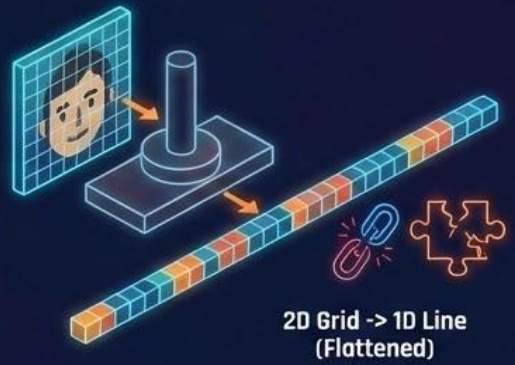
Why Standard Networks Fail at Images

The Weight Explosion



A standard 1080p color image contains over 6 million pixels. If we use a fully connected layer where every neuron connects to every pixel, the number of parameters reaches the billions immediately.

Loss of Spatial Structure



To feed an image into a standard dense layer, we must "flatten" the 2D grid into a 1D line. This destroys the spatial relationship between neighboring pixels.

Lack of Translation Invariance

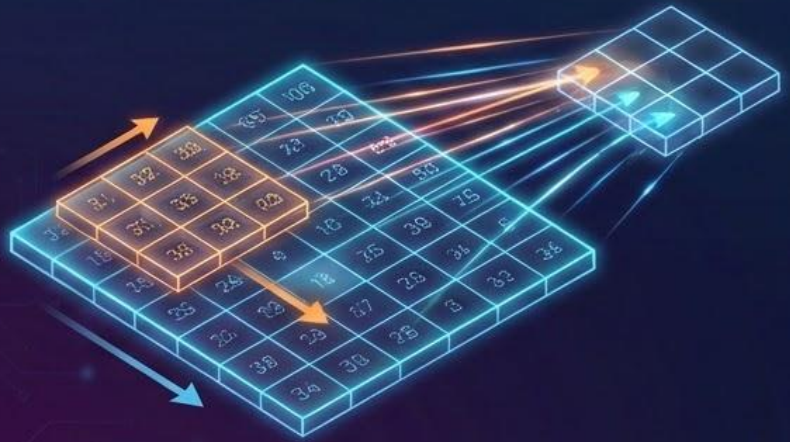
If an object (like a cat) appears in the top-left of the training image, but in the bottom-right of the testing image, a fully connected network treats it as an entirely new, unrecognized pattern.

The Convolution Operation

Sliding Filters over Pixels

The Solution:

Instead of connecting to every pixel at once, we apply small filters (also called kernels) that slide systematically across the input image to detect local patterns.



The Mathematics:

$$(I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

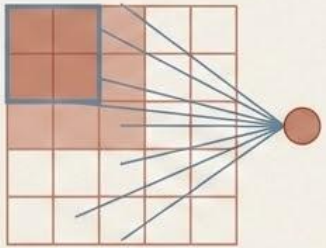
- I: The input image matrix.
- K: The Kernel/Filter (e.g., a 3×3 matrix of weights).

Output:
A smaller matrix known as a 'Feature Map.'

Why Convolution Works

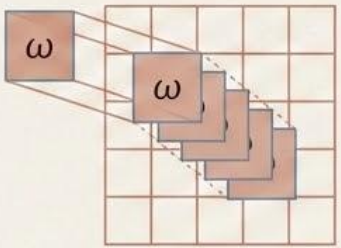
Efficiency and Pattern Recognition

Local Connectivity



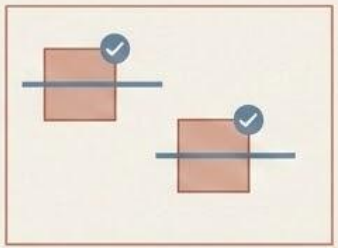
Each output node is not connected to the whole image; it only connects to a small, localized region of the input (its receptive field). This preserves spatial structure.

Parameter Sharing



The exact same filter (the same 3 × 3 set of weights) is applied everywhere across the image. This reduces millions of weights down to just $9.

Translation Invariance

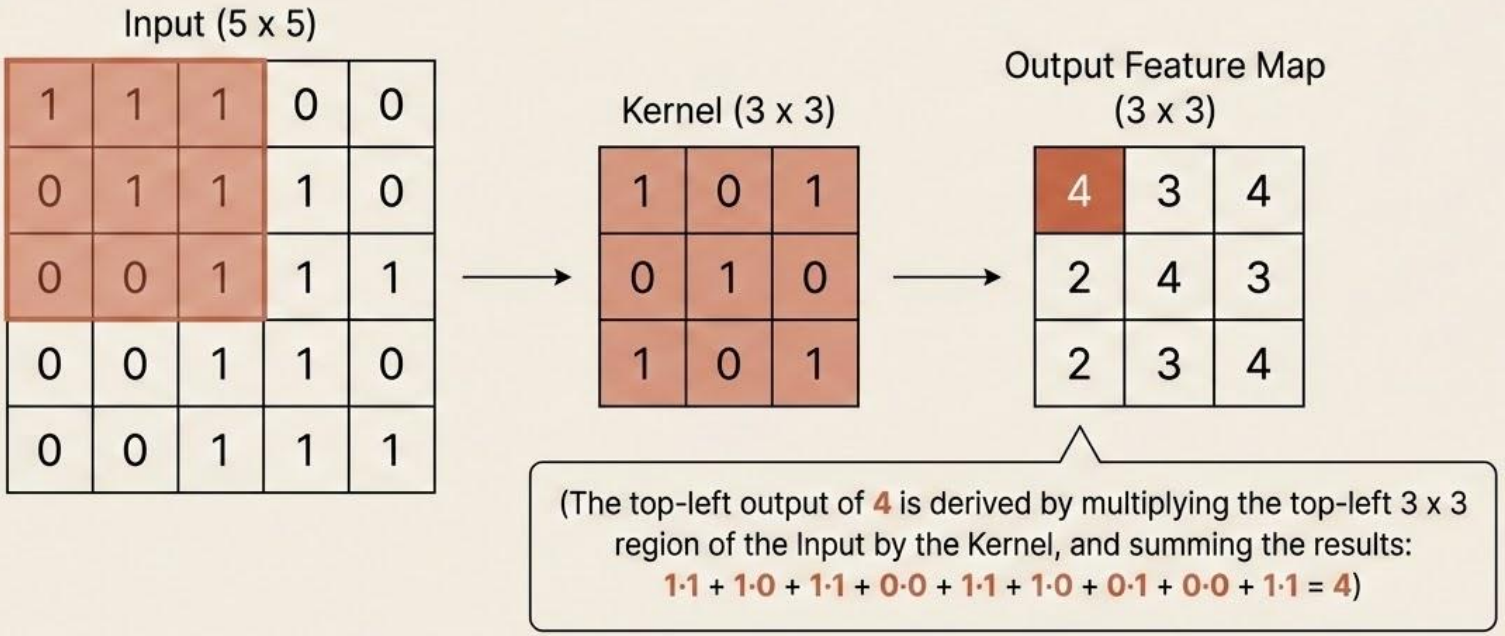


Because the same filter slides over the entire image, if it is designed to detect a horizontal edge, it will successfully detect that edge regardless of where it appears on the screen.

A Concrete Example

Applying a 3 x 3 Kernel

Let's look at the actual numbers when applying a 3 x 3 kernel to a 5 x 5 input image.



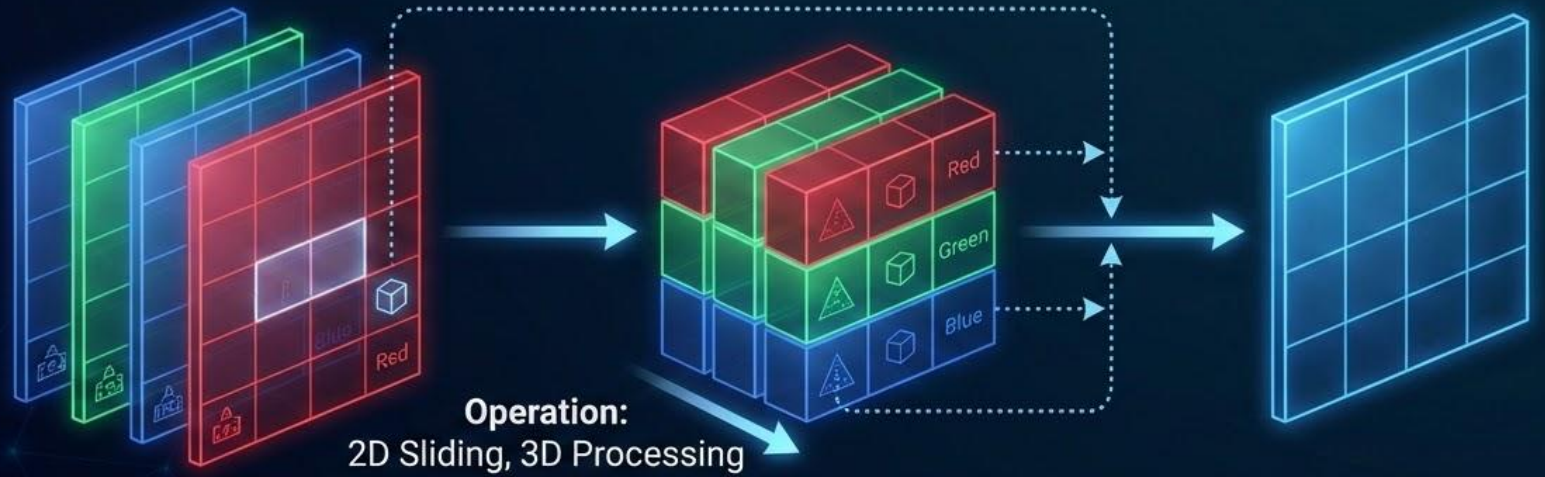
Convolution for Color (RGB) Images

Visualizing 3D Input Processing

RGB Input
(3D Volume)

3 × 3 × 3 Filter
(Matches Input Depth)

Output
(2D Feature Map)



Operation:
2D Sliding, 3D Processing

Height × Width × 3 Channels

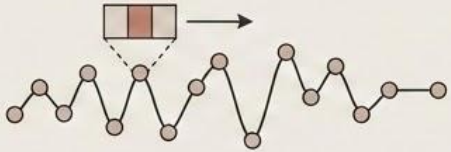
3 × 3 × 3 Filter
(Matches Input Depth)

Output from one filter is a single 2D feature map (matrix).

1D and 3D Convolution

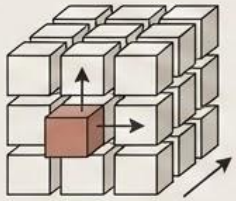
Beyond Static 2D Images

1D Convolution (Sequences)



- **Input Type:** Sequential data such as text (sentences, where word embeddings provide the secondary dimension) or time series (like stock prices over time).
- **Mechanism:** The filter has a single dimension (length) and slides along the time or sequence dimension.
- **Output:** A sequence (or 1D matrix) of features.

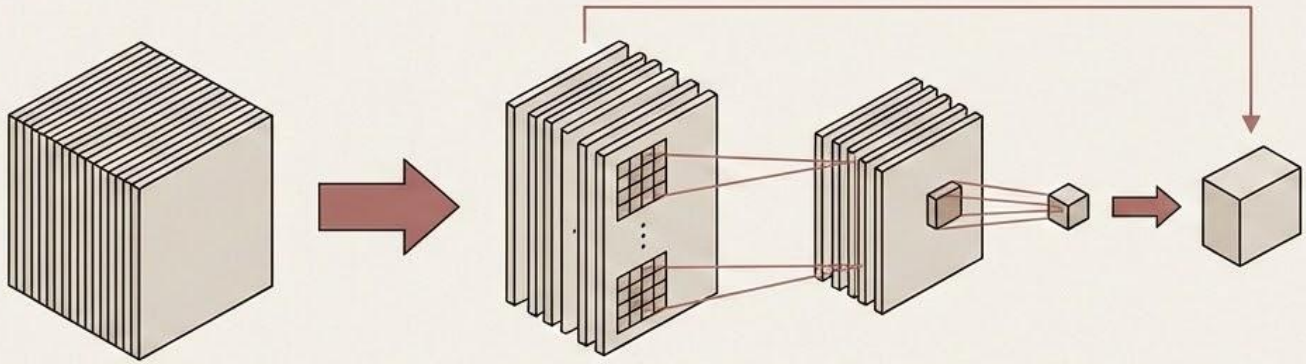
3D Convolution (Volumetric)



- **Input Type:** True volumetric data like medical scans (CT, MRI volumes with height × width × depth) or videos (height × width × time, where time is the third dimension).
- **Mechanism:** Filters have three dimensions (e.g., 3 × 3 × 3) and slide in all three dimensions simultaneously.
- **Output:** A 3D volumetric feature map.

Depthwise Separable Convolution

The Mobile Architecture Workhorse



Standard Convolution
(Heavy)

1. Depthwise Convolution
(Spatial, per channel)

2. Pointwise Convolution
(1×1, Channel Mixing)

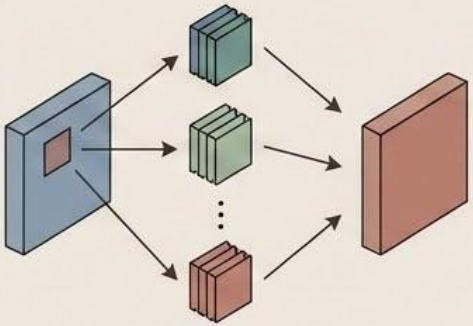
The Problem: Standard convolution is parameter-heavy and slow on mobile devices.

Benefit: Massive reduction in parameters, enabling high-performance deep learning on mobile hardware.

Anatomy of a CNN Layer

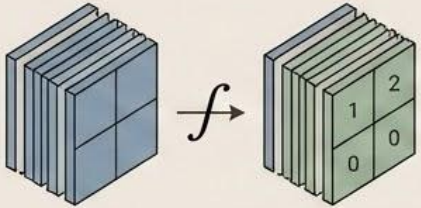
The Convolutional Layer: The Three-Step Process

1. Convolution



- Apply multiple filters to the input.
- **Output:** Each filter produces one feature map. The number of output channels equals the number of filters used.

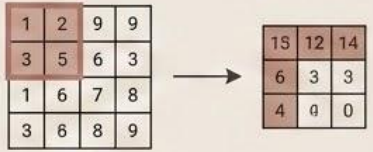
2. Activation (Typically ReLU)



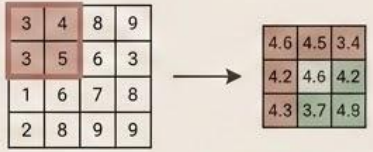
$$f(x) = \max(0, x)$$

Applies non-linearity to the feature maps, allowing the network to learn complex patterns without vanishing gradients.

3. Pooling (Downsampling)



Max Pooling: Takes the highest value in a given window (keeps the strongest signal).



Average Pooling: Takes the mathematical average.

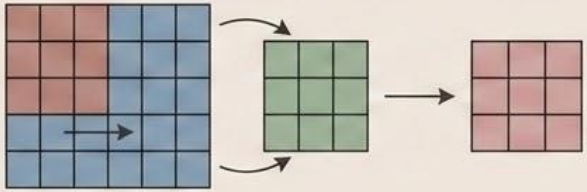
Purpose: Drastically reduces spatial dimensions (saving memory) and increases the receptive field of subsequent layers.

Stride and Padding

Navigating the Input Image

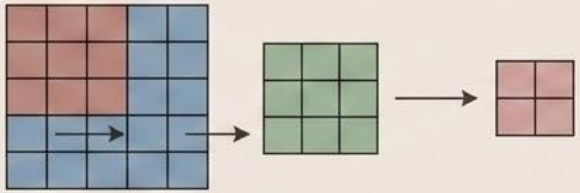
Stride (S):

- **Stride 1:**



The filter moves 1 pixel at a time. The output size remains roughly equal to the input size.

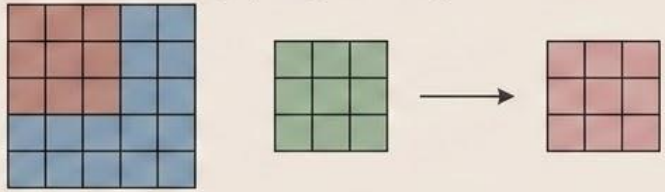
- **Stride 2:**



The filter skips a pixel with every step. The spatial output size is cut in half.

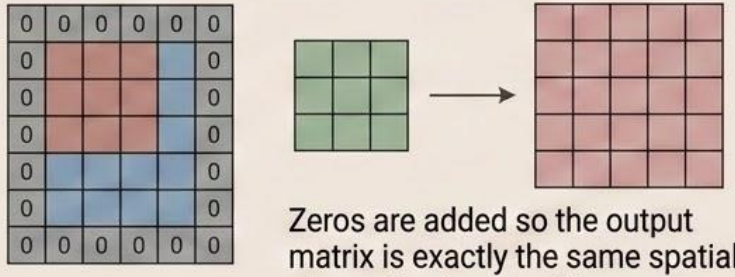
Padding (P):

- **Valid Padding (No padding):**



The filter cannot go over the edge. The output will shrink.

- **Same Padding (Zero padding):**



Zeros are added so the output matrix is exactly the same spatial size as the input matrix.

The Output Size Formula

Calculating Network Dimensions

Knowing the exact spatial size of your data at every layer is critical for building CNN architectures.

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

- O = Output dimension (Width or Height)
- W = Input dimension size (Width or Height)
- K = Kernel/Filter size
- P = Padding size
- S = Stride step size

Note: We take the “floor” of the division if it is not an integer.

The Full Architecture

From Pixels to Predictions

Input Image: $224 \times 224 \times 3$ (RGB) 

Feature Extraction:

[Conv 3x3, 64 filters] + ReLU + MaxPool (2x2)

[Conv 3x3, 128 filters] + ReLU + MaxPool (2x2)

[Conv 3x3, 256 filters] + ReLU + MaxPool (2x2)

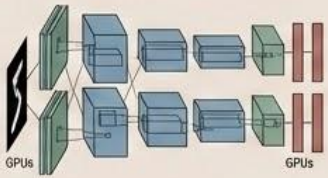
Flattening: Convert the final 3D feature maps into a 1D vector.

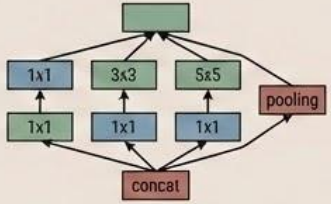
Classification Head: Fully Connected Layers → Softmax Output

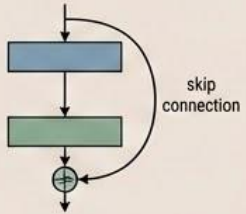
Famous CNN Architectures

The Evolution of Computer Vision

1998 — 2012 — 2014 — 2014 — 2015







LeNet-5 (1998)

The foundational CNN, designed for handwritten digit recognition (reading zip codes).

AlexNet (2012)

The model that won ImageNet, utilized GPUs, and officially kicked off the modern deep learning boom.

VGG (2014)

Introduced a very deep but mathematically simple architecture, proving that stacking many small (3x3) convolutions is better than using large ones.

Inception (2014)

Introduced complex, parallel "Inception blocks" applying multiple filter sizes simultaneously.

ResNet (2015)

Invented skip connections, allowing engineers to train networks with over 100 layers without vanishing gradients.

The Sequential Data Problem

Why Feedforward Networks Fall Short

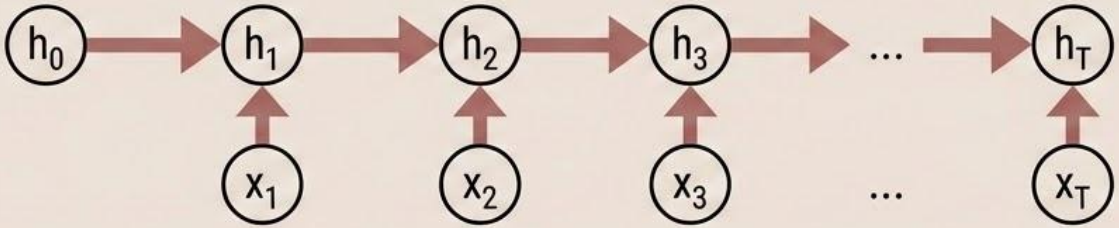
The Problem

- Standard feedforward and convolutional networks assume all inputs are independent of each other.
- They cannot handle data where order matters (like words in a sentence).
- They struggle with inputs of varying lengths (like an audio clip).

The Solution (RNN)

- Recurrent Neural Networks introduce a “loop” in the architecture.
- They maintain a hidden state (memory) that gets passed from one time step to the next.
- This allows the network to persist information across the sequence.

The Unrolled View



(The network processes input x_t while remembering context h_{t-1})

The Math of Memory

The RNN Equations

Updating the Hidden State

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Combines the current input (x_t) with the previous memory (h_{t-1}).

$$y_t = W_{hy}h_t + b_y$$

Calculates a prediction based on the newly updated memory.

Variables

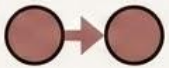
- h_t = Hidden state (memory) at time t
- x_t = Input at time t
- y_t = Output at time t
- W and b = Learned weights and biases

RNN Architectures

Mapping Inputs to Outputs

RNNs are highly flexible and can be “wired” differently depending on the task:

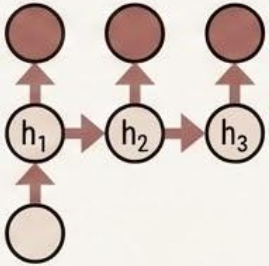
One-to-One



Input: Single
Output: Single

Standard Neural Network (e.g., Image Classification)

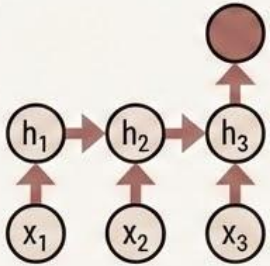
One-to-Many



Input: Single
Output: Sequence

Image Captioning (1 image → sentence of words)

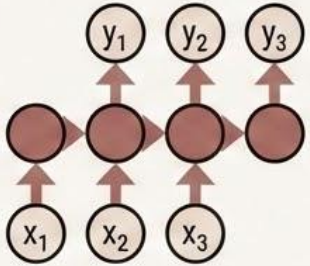
Many-to-One



Input: Sequence
Output: Single

Sentiment Analysis (Sentence → “Positive/Negative”)

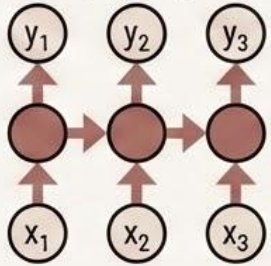
Many-to-Many



Input: Sequence
Output: Sequence

Machine Translation (English sentence → French sentence)

Many-to-Many (Sync)



Input: Sequence
Output: Sequence

Video Classification (Frame by frame analysis)

Real-World Applications

Where Sequences Matter

Natural Language Processing (NLP)

- Language modeling (predicting the next word, like autocomplete).
- Machine translation and Sentiment analysis.

Time Series Analysis

- Stock price prediction and economic forecasting.
- Anomaly detection in server logs or IoT devices.

Speech Recognition

- Converting audio waveforms into text (Siri, Alexa).
- Speaker identification.

Video Analysis

- Action recognition (what is happening in this clip?).
- Frame prediction (what happens next?).

Limitations of Basic RNNs

Why We Needed to Evolve

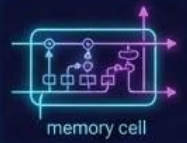
The Fatal Flaw: Basic RNNs suffer from **extreme short-term memory**. If a sequence is too long, they completely forget the beginning of the sentence by the time they reach the end.

The Mathematical Cause:

Vanishing Gradients: As backpropagation moves backward through time, the gradients shrink exponentially. The network cannot learn long-range dependencies.

Exploding Gradients: Conversely, gradients can multiply out of control, breaking the model.

The Modern Solutions (Advanced RNNs):



LSTM (Long Short-Term Memory)
Uses complex "gates" to explicitly decide what to remember and what to forget.

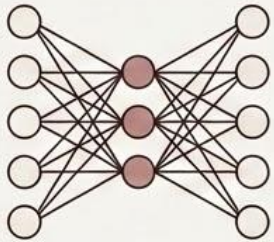
GRU (Gated Recurrent Unit)
A streamlined, faster version of the LSTM.

Attention Mechanisms
Allows the network to "look back" at specific parts of the sequence, bypassing the bottleneck entirely (leading to Transformers).

The Right Tool for the Job

Architecture Comparison: Matching Networks to Data

Autoencoder

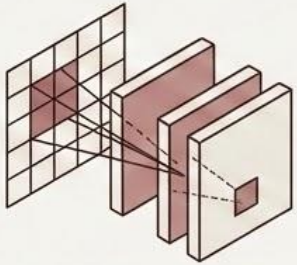


Best For: Unsupervised learning, data compression.

Key Feature: The Bottleneck layer.

Complexity: Moderate

CNN

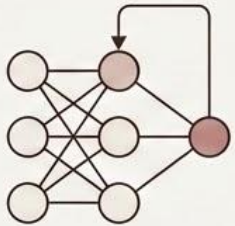


Best For: Images, spatial and grid data.

Key Feature: Local connectivity, weight sharing.

Complexity: Moderate

RNN

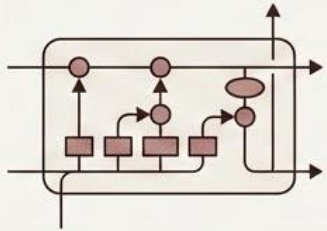


Best For: Sequences, short time series.

Key Feature: Hidden state, recurrence.

Complexity: Low-Moderate

LSTM



Best For: Long sequences, complex time series.

Key Feature: Gated memory.

Complexity: Moderate

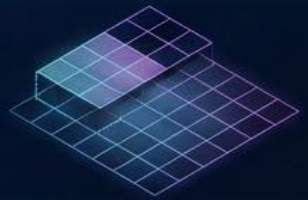
Key Formulas to Remember

The Math Behind the Architectures

Categorical Cross-Entropy (The Loss Function)

$$L = -\sum y_i \log(\hat{y}_i)$$

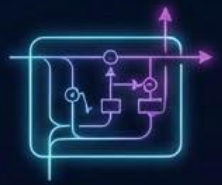
(Penalizes confident wrong predictions in classification.)



The Convolution Operation (CNNs)

$$(I * K)(i,j) = \sum_m \sum_n I(+m, j+n) K(m,n)$$

(The sliding window that extracts spatial features.)



The RNN State Update (Memory)

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

(Combining new input with the memory of the past.)

Core Insights

Making Deep Learning Practical

- **Pretraining / Transfer Learning:** Enables the training of massive deep learning models even when you have extremely limited labeled data.
- **Cross-Entropy:** The absolute standard loss function for classification tasks.
- **Autoencoders:** Prove that neural networks can learn powerful, foundational representations of the world without human labels.
- **CNNs:** Revolutionized computer vision by exploiting spatial structure and sharing parameters to run efficiently.
- **RNNs:** Allowed AI to process time and sequences, though they required the invention of LSTMs and Attention to overcome the vanishing gradient problem.